

Operadores en Python

Capítulo técnico, educativo y práctico

Curso Curso de Python	Módulo / Unidad Capítulo 4
Título Operadores	Nivel Desde principiante hasta nivel práctico
Enfoque Desarrollo de Software Python	Meta Comprender, analizar y aplicar operadores

Idea central: los operadores permiten calcular, comparar, combinar, buscar, verificar identidad y manipular información. Son el puente entre los datos almacenados y la lógica que hace que un programa sea útil.

1. Objetivo del capítulo

Este capítulo enseña los operadores de Python desde cero. El estudiante comenzará identificando operandos y expresiones, después aprenderá cada familia de operadores y finalmente integrará varias familias en problemas de software e industria.

Al finalizar este capítulo serás capaz de:

- Explicar qué es un operador, un operando y una expresión.
- Usar operadores aritméticos para resolver cálculos.
- Actualizar variables mediante asignación compuesta.
- Comparar valores con resultados booleanos.
- Combinar condiciones con `and`, `or` y `not`.
- Comprobar pertenencia con `in` y `not in`.
- Distinguir igualdad de identidad: `==` frente a `is`.
- Reconocer operadores bit a bit y su aplicación en sistemas de bajo nivel.
- Resolver expresiones respetando precedencia y utilizando paréntesis cuando convenga.
- Aplicar operadores en inventarios, sensores, producción, máquinas y sistemas empresariales.

2. Introducción al tema

¿Qué es un operador?

Un operador es un símbolo o palabra que indica una operación sobre uno o más valores. `+` indica suma, `>` una comparación y `and` combina condiciones.

Una analogía sencilla es una calculadora: los números son los datos y los botones indican qué hacer. En Python, los operadores expresan esas acciones dentro de las expresiones.

¿Por qué existe?

Los programas necesitan transformar datos y evaluar reglas. Un sistema industrial puede preguntar si una temperatura supera un límite; un inventario puede calcular existencias; una aplicación puede comprobar si un usuario tiene un permiso. Los operadores expresan estas relaciones de forma precisa.

¿Qué ocurriría sin ellos?

Sería extremadamente difícil expresar cálculos, comparaciones y reglas de decisión. Las variables podrían conservar información, pero el programa tendría pocas herramientas para transformarla o evaluarla.

Relación con el curso

Los operadores se apoyan en variables, tipos de datos, entrada y salida. Después serán fundamentales para `if/elif/else`, `while`, `for` y funciones.

3. Fundamentos: operandos, operadores y expresiones

Concepto	Definición sencilla	Ejemplo
Operador	Indica qué operación realizar.	+
Operando	Dato sobre el que actúa el operador.	10
Expresión	Combinación que Python evalúa.	10 + 5
Resultado	Valor obtenido al evaluar.	15

```
a = 10
b = 5
resultado = a + b
print(resultado)
```

Paso a paso: a y b son variables; + es el operador; a + b es la expresión; 15 es el resultado y se almacena en resultado.

4. Mapa general de operadores

Familia	Operadores	Propósito
Aritméticos	+ - * / // % **	Cálculos.
Asignación	= += -= *= /= //= %= **=	Crear y actualizar variables.
Comparación	== != > < >= <=	Comparar valores.
Lógicos	and or not	Combinar o negar condiciones.
Pertenencia	in, not in	Comprobar contenido o pertenencia.
Identidad	is, is not	Comprobar si es el mismo objeto.
Bit a bit	& ^ ~ << >>	Operar sobre bits de enteros.

Regla mental: aritmética calcula → asignación guarda → comparación pregunta → lógica combina → pertenencia busca → identidad verifica → bit a bit trabaja con representación binaria.

5. Operadores aritméticos

¿Qué son?

Son operadores para cálculos numéricos. Se utilizan en producción, inventarios, promedios, conversiones, mediciones y muchas otras tareas.

Operador	Nombre	Ejemplo	Resultado
+	Suma	10 + 3	13
-	Resta	10 - 3	7
*	Multiplicación	10 * 3	30
/	División	10 / 4	2.5
//	División entera	10 // 4	2
%	Resto	10 % 4	2
**	Potencia	2 ** 3	8

Ejemplo básico

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

La línea de cada `print()` evalúa una expresión diferente. Observa que `/` y `//` tienen propósitos distintos.

Ejemplo industrial

```
piezas = 53
por_caja = 10

cajas = piezas // por_caja
sobrantes = piezas % por_caja

print("Cajas completas:", cajas)
print("Sobrantes:", sobrantes)
```

Resultado: 5 cajas completas y 3 piezas sobrantes. `//` obtiene los grupos completos y `%` el resto.

5.1 Ejemplos progresivos de aritmética

Nivel 1 — Básico

```
precio = 100
cantidad = 3
total = precio * cantidad
print(total)
```

Se multiplican dos variables para obtener un resultado. Error típico: usar texto donde se espera un número.

Nivel 2 — Intermedio

```
produccion = 850
defectuosas = 27
buenas = produccion - defectuosas
print("Piezas buenas:", buenas)
```

Se conservan los datos originales y se crea una variable derivada.

Nivel 3 — Práctico

```
horas = 8
piezas_por_hora = 125
produccion = horas * piezas_por_hora
objetivo = 1200
pendientes = objetivo - produccion

print("Producción:", produccion)
print("Pendientes:", pendientes)
```

El programa representa una regla de producción: producción = horas × capacidad por hora. Si la producción supera el objetivo, la variable de pendientes puede ser negativa; el significado de ese caso debe decidirse según el problema.

Nivel 4 — Profesional

```
stock_inicial = 500
entradas = 120
salidas = 185

stock_actual = stock_inicial + entradas - salidas
print("Stock actual:", stock_actual)
```

Este patrón puede integrarse después con validaciones, funciones y estructuras de datos.

6. Operadores de asignación

¿Qué son?

= asigna un valor. Los operadores compuestos combinan una operación con una asignación, lo que resulta útil para actualizar contadores y acumuladores.

Operador	Equivalencia conceptual	Uso
=	$x = \text{valor}$	Crear/asignar.
+=	$x = x + \text{valor}$	Incrementar.
-=	$x = x - \text{valor}$	Reducir.
*=	$x = x * \text{valor}$	Escalar.
/=	$x = x / \text{valor}$	Dividir y actualizar.
//=	$x = x // \text{valor}$	División entera.
%=	$x = x \% \text{valor}$	Actualizar con resto.
**=	$x = x ** \text{valor}$	Potenciar.

```
contador = 0
contador += 1
contador += 1
contador -= 1
print(contador)
```

Evolución del valor: $0 \rightarrow 1 \rightarrow 2 \rightarrow 1$. El mismo patrón aparece en contadores de piezas, ciclos y registros.

Error de confusión

= no pregunta si dos valores son iguales. Para comparar se utiliza ==. Esta diferencia será esencial en condicionales.

7. Operadores de comparación

¿Qué son?

Permiten comparar valores. Las comparaciones normales producen `True` o `False`, valores booleanos que después pueden utilizarse para tomar decisiones.

Operador	Pregunta	Ejemplo
<code>==</code>	¿Son iguales?	<code>temperatura == 80</code>
<code>!=</code>	¿Son diferentes?	<code>estado != "OFF"</code>
<code>></code>	¿Izquierda es mayor?	<code>nivel > limite</code>
<code><</code>	¿Izquierda es menor?	<code>nivel < limite</code>
<code>>=</code>	¿Mayor o igual?	<code>edad >= 18</code>
<code><=</code>	¿Menor o igual?	<code>velocidad <= maxima</code>

```
temperatura = 85
limite = 80
```

```
supera_limite = temperatura > limite
print(supera_limite)
```

Análisis: Python obtiene 85 y 80, aplica `>`, obtiene `True` y guarda ese resultado. Esta es la base de una condición.

Ejemplo de sensor

```
velocidad_actual = 1750
velocidad_maxima = 1800

dentro_del_limite = velocidad_actual <= velocidad_maxima
print("Dentro del límite:", dentro_del_limite)
```

El nombre `dentro_del_limite` comunica el significado del booleano y mejora la legibilidad.

8. Operadores lógicos

¿Qué son?

Permiten combinar condiciones. `and` requiere que las dos condiciones sean verdaderas; `or` necesita que al menos una sea verdadera; `not` niega una expresión.

A	B	A and B	A or B
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Ejemplo: seguridad de máquina

```
motor_activo = True
puerta_cerrada = True

puede_operar = motor_activo and puerta_cerrada
print(puede_operar)
```

Ambas condiciones son verdaderas, por lo que `puede_operar` resulta `True`.

Ejemplo con OR

```
alarma_temperatura = True
alarma_presion = False

hay_alarma = alarma_temperatura or alarma_presion
print(hay_alarma)
```

Existe al menos una alarma activa, por lo que el resultado es verdadero.

Ejemplo con NOT

```
sensor_falla = False
sensor_disponible = not sensor_falla
print(sensor_disponible)
```

`not` invierte el valor lógico. La expresión produce `True`.

8.1 Expresiones lógicas más realistas

```
temperatura = 72.5
limite = 80.0
sensor_activo = True

temperatura_segura = temperatura <= limite
sistema_ok = sensor_activo and temperatura_segura

print("Temperatura segura:", temperatura_segura)
print("Sistema OK:", sistema_ok)
```

Aquí ocurre una cadena muy importante: dato → comparación → condición → combinación lógica. Más adelante, un `if` podrá utilizar `sistema_ok` para decidir qué hacer.

Errores frecuentes con lógica

- Usar `and` cuando basta una condición: puede producir un resultado demasiado restrictivo.
- Usar `or` cuando todas deben cumplirse: puede permitir situaciones no deseadas.
- Construir expresiones enormes sin paréntesis ni nombres intermedios.
- No considerar los casos límite, como temperatura exactamente igual al límite.

Buenas prácticas

Cuando una expresión representa una regla importante, divide el problema en condiciones con nombres descriptivos. Esto facilita leer, probar y modificar el programa.

9. Operadores de pertenencia: in y not in

¿Qué son?

Comprueban si un elemento pertenece a una secuencia o colección, por ejemplo una cadena, lista, tupla o conjunto.

```
maquinas = ["CNC-01", "CNC-02", "TORNO-01"]

print("CNC-01" in maquinas)
print("ROBOT-01" not in maquinas)
```

La primera pregunta es si CNC-01 está dentro de la lista. La segunda pregunta si ROBOT-01 no está dentro.

Con cadenas

```
modelo = "Python"

print("Py" in modelo)
print("Java" not in modelo)
```

En una cadena, in comprueba si existe la subcadena indicada.

Caso práctico: roles

```
rol = "operador"
roles_permitidos = ["operador", "supervisor", "ingeniero"]

autorizado = rol in roles_permitidos
print(autorizado)
```

Este patrón sirve como ejemplo de validación. En aplicaciones reales, una lista local no debe considerarse por sí sola un sistema completo de autorización.

10. Identidad: is e is not

`==` pregunta si los valores son iguales; `is` pregunta si dos nombres se refieren al mismo objeto. Esta diferencia es fundamental al trabajar con objetos.

Operador	Pregunta	Uso típico
<code>==</code>	¿Tienen el mismo valor?	Comparaciones.
<code>is</code>	¿Es el mismo objeto?	Identidad; especialmente <code>None</code> .
<code>is not</code>	¿No es el mismo objeto?	Identidad negativa.

```
lista_a = [1, 2, 3]
lista_b = [1, 2, 3]
lista_c = lista_a

print(lista_a == lista_b)
print(lista_a is lista_b)
print(lista_a is lista_c)
```

Resultados: `True`, `False`, `True`. A y B tienen el mismo contenido pero son listas distintas; C es otro nombre para el mismo objeto que A.

Caso recomendado: None

```
resultado = None

if resultado is None:
    print("No hay resultado")
```

Para comprobar `None`, utiliza `is None`. No uses `is` como sustituto general de `==`.

11. Operadores bit a bit

Son operadores que trabajan sobre bits de enteros. Son más avanzados y aparecen en sistemas embebidos, protocolos, máscaras de bits, controladores y programación de bajo nivel.

Operador	Nombre	Idea
&	AND	1 solo si ambos bits son 1.
	OR	1 si al menos un bit es 1.
^	XOR	1 si son diferentes.
~	NOT	Invierte bits según la representación de enteros de Python.
<<	Desplazamiento izq.	Desplaza bits a la izquierda.
>>	Desplazamiento der.	Desplaza bits a la derecha.

```
a = 5      # 0101
b = 3      # 0011

print(a & b) # 0001 -> 1
print(a | b) # 0111 -> 7
print(a ^ b) # 0110 -> 6
```

Ejemplo conceptual: banderas de estado

```
MOTOR = 1
ALARMA = 2
PUERTA = 4

estado = MOTOR | ALARMA
print(estado)
```

Aquí se combinan dos banderas dentro de un entero. El concepto es útil para representar varios estados en bits, pero requiere una definición clara de cada máscara.

Recomendación

No necesitas dominar bit a bit antes de avanzar en tu curso básico. Aprende primero aritmética, asignación, comparación y lógica; después profundiza en binario cuando el problema lo requiera.

12. Precedencia y orden de evaluación

Cuando una expresión contiene varios operadores, Python debe determinar qué se evalúa primero. La precedencia establece ese orden.

```
resultado = 2 + 3 * 4
print(resultado)
```

Resultado: 14. Primero se realiza la multiplicación y después la suma.

```
a = 2 + 3 * 4
b = (2 + 3) * 4

print(a)
print(b)
```

Los resultados son 14 y 20. Los paréntesis permiten cambiar el orden y, además, hacen explícita la intención.

Prioridad aproximada	Elementos
Alta	()
	**
	+X, -X, ~X
	* / // %
	+ -
	< <= > >= == !=
	not
	and
Baja	or

Consejo profesional: cuando una expresión sea difícil de leer, utiliza paréntesis o variables intermedias. No dependas de que otra persona recuerde toda la tabla de precedencia.

13. Ejemplos integradores

Nivel 1 — Variables + aritmética

```
precio = 100
cantidad = 4
total = precio * cantidad
print("Total:", total)
```

El operador `*` transforma dos datos en un resultado.

Nivel 2 — Entrada + conversión + operadores

```
precio = float(input("Precio: "))
cantidad = int(input("Cantidad: "))

total = precio * cantidad
print("Total:", total)
```

Como `input()` devuelve texto, se convierte antes de calcular. Este ejemplo conecta el capítulo anterior de entrada/salida con operadores.

Nivel 3 — Sensor industrial

```
temperatura = float(input("Temperatura °C: "))
limite = 80.0
sensor_activo = True

temperatura_segura = temperatura <= limite
sistema_ok = sensor_activo and temperatura_segura

print("Temperatura:", temperatura)
print("Sistema OK:", sistema_ok)
```

Se combinan conversión, comparación y lógica. Todavía no se ejecuta una acción de control; el booleano queda preparado para el próximo capítulo de decisiones.

13.1 Nivel 4 — Inventario profesional

```
producto = input("Producto: ")
precio_unitario = float(input("Precio unitario: "))
cantidad = int(input("Cantidad: "))
descuento = float(input("Descuento decimal: "))

subtotal = precio_unitario * cantidad
monto_descuento = subtotal * descuento
total = subtotal - monto_descuento

print("\n--- RESUMEN ---")
print("Producto:", producto)
print("Cantidad:", cantidad)
print("Subtotal:", subtotal)
print("Descuento:", monto_descuento)
print("Total:", total)
```

Análisis línea por línea

- `producto` conserva texto.
- Los números se convierten a `float` o `int` según corresponda.
- `subtotal` usa multiplicación.
- `monto_descuento` usa multiplicación.
- `total` usa resta.
- `print()` presenta los resultados.

Error relacionado

Si el usuario introduce 15 pensando en «15 %» pero el programa espera 0.15, el cálculo será incorrecto. En software real, define claramente el formato de entrada y valida los valores.

Nota profesional sobre dinero

Los ejemplos introductorios usan `float`. En sistemas financieros, estudia `decimal.Decimal` y reglas explícitas de redondeo para evitar tratar el punto flotante como exactitud decimal garantizada.

14. Errores de confusión comunes

Confusión	Qué significa	Cómo recordarlo
= / ==	Asignación / igualdad.	Guardar vs preguntar.
== / is	Valor / identidad.	Mismo contenido vs mismo objeto.
//	División normal / división entera.	Cociente real vs cociente entero.
// %	División / resto.	Cociente vs lo que queda.
and / or	Todas / al menos una.	Ambas vs cualquiera.
in / ==	Pertenencia / igualdad.	Está dentro vs es igual.
not / !=	Negación / diferencia.	Negar expresión vs comparar.

15. Errores comunes al trabajar con operadores

1. Confundir = con ==

```
# Comparación correcta
if temperatura == 80:
    print("Exactamente 80")
```

La comparación usa ==. Confundirlo con = es especialmente frecuente al comenzar con `if`.

2. Dividir entre cero

Una división con divisor cero produce un error. Si el divisor puede venir del usuario o de datos externos, valida ese caso antes del cálculo.

3. Mezclar tipos sin intención

```
edad = "20"
# edad + 5 -> error de tipos
```

El texto y el entero no son intercambiables automáticamente para esta operación. Convierte el dato cuando corresponda.

4. Condiciones demasiado complejas

Divide una regla grande en condiciones con nombres descriptivos o usa paréntesis para que la intención sea evidente.

16. Buenas prácticas

- **Nombres descriptivos:** `temperatura_segura` comunica más que `x`.
- **Paréntesis con intención:** aclaran expresiones complejas y reducen errores de interpretación.
- **Variables intermedias:** facilitan pruebas y depuración cuando una fórmula tiene varios pasos.
- **Unidades explícitas:** nombres como `velocidad_rpm` o `temperatura_c` reducen ambigüedad.
- **Conserva datos originales:** crea resultados derivados en lugar de sobrescribir información que aún necesitas.
- **Valida entradas:** los operadores no corrigen por sí solos datos inválidos.
- **Usa `==` para valores e `is` para identidad:** especialmente importante con `None`.
- **Prueba casos límite:** cero, negativos, valores máximos, listas vacías y datos inesperados.
- **Prioriza claridad:** no hagas una expresión corta si una versión ligeramente más larga es mucho más fácil de mantener.

Organización, mantenimiento y escalabilidad

En un proyecto pequeño puedes escribir una expresión directamente. Cuando el programa crece, conviene separar responsabilidades: obtener datos, calcular resultados, validar condiciones y presentar información. Esta organización facilita reutilizar la lógica dentro de funciones y probar cada parte.

Seguridad y rendimiento

La seguridad depende del diseño completo del sistema, no de un operador aislado. Para rendimiento, primero busca corrección y claridad; optimiza después de medir un problema real.

17. Conectando conocimientos

Tema anterior	Cómo se conecta
Variables	Los operadores leen y transforman valores.
Tipos de datos	El tipo influye en qué operaciones son válidas.
Entrada	Los datos recibidos pueden convertirse antes de operar.
Salida	<code>print()</code> muestra resultados de expresiones.
Siguiente: decisiones	Comparaciones y lógica forman condiciones.
Bucles	Asignación y aritmética actualizan contadores y acumuladores.
Funciones	Las expresiones forman parte de cálculos reutilizables.

Cadena de aprendizaje: entrada → variable → operador → resultado → condición → decisión → repetición → aplicación.

18. Ejercicios prácticos

No se incluyen soluciones junto al ejercicio. Intenta resolver cada problema primero y utiliza la autoevaluación para comprobar conceptos.

Ejercicio 1 — Comprensión básica

Objetivo: practicar aritmética. **Enunciado:** crea variables para precio y cantidad; calcula subtotal, descuento y total.

Requisitos: usa multiplicación y resta. **Pista:** conserva los datos originales.

Ejercicio 2 — Aplicación

Contexto: una máquina produce 37 piezas por ciclo durante 14 ciclos. Calcula producción total y grupos completos de 20 piezas, además de sobrantes. **Restricciones:** usa *, // y %. **Pista:** calcula primero la producción total.

Ejercicio 3 — Integración

Problema: pide temperatura y estado de sensor. Determina si la temperatura está dentro del límite y si el sistema puede operar. **Requisitos:** entrada, conversión, comparación y lógica. **Pista:** crea condiciones independientes antes de combinarlas.

Ejercicio 4 — Caso práctico

Inventario: registra stock actual, stock mínimo y unidades recibidas. Calcula el nuevo stock y determina si queda por debajo del mínimo. **Requisitos:** usa aritmética y comparación. **Pista:** separa «nuevo stock» de «stock bajo».

Ejercicio 5 — Reto

Problema: diseña la lógica inicial de un monitor de producción con piezas producidas, defectuosas, objetivo, temperatura y sensor. **Funcionalidades:** piezas buenas, pendientes, porcentaje de defectos, temperatura segura y estado general. **Reglas:** evita división por cero; usa al menos cuatro familias de operadores; conserva datos originales. **Pistas:** 1) identifica entradas; 2) calcula derivados; 3) crea condiciones; 4) combina condiciones; 5) prueba casos límite.

19. Mini proyecto del capítulo

Nombre del proyecto

Monitor básico de una estación de producción

Contexto

Una estación industrial necesita un programa de consola que reciba información y genere un diagnóstico básico. No se conecta todavía a un PLC real: se simula el comportamiento con datos introducidos por el usuario.

Objetivo

Aplicar variables, entrada/salida y operadores aritméticos, de asignación, comparación y lógica para transformar datos en información útil.

Requisitos funcionales

- Solicitar nombre de la máquina.
- Solicitar piezas producidas y defectuosas.
- Solicitar objetivo de producción.
- Solicitar temperatura y límite.
- Representar el estado del sensor.
- Calcular piezas buenas y pendientes.
- Determinar temperatura segura.
- Determinar disponibilidad general.
- Mostrar un reporte final.

Requisitos técnicos

- Usar nombres descriptivos en snake_case.
- Convertir correctamente datos numéricos.
- Usar al menos cuatro familias de operadores.
- Evitar división por cero al calcular porcentajes.
- No sobrescribir datos originales innecesariamente.

Paso 1 — Modelar

Haz una tabla con cada dato, su variable, su tipo y su significado. Todavía no escribas los cálculos.

Paso 2 — Capturar

Obtén los datos con `input()` y realiza las conversiones adecuadas. Piensa qué ocurre si el usuario introduce cero o un valor extremo.

Paso 3 — Calcular y comparar

Construye resultados derivados y condiciones independientes. Prueba cada expresión con valores conocidos.

Paso final — Integrar

Combina las condiciones para formar un diagnóstico y genera un reporte claro. Después prueba casos normales y casos límite.

20. Comprueba tus conocimientos

- 1. ¿Qué es un operador y qué es un operando?
- 2. ¿Qué diferencia hay entre una expresión y su resultado?
- 3. ¿Qué produce $10 / 4$?
- 4. ¿Qué diferencia existe entre $/$ y $//$?
- 5. ¿Para qué sirve $\%$? Da un ejemplo industrial.
- 6. ¿Qué significa $+=$?
- 7. ¿Qué resultado producen normalmente las comparaciones?
- 8. Diferencia $==$ e is .
- 9. ¿Cuándo usarías `and` en una máquina?
- 10. ¿Cuándo usarías `or`?
- 11. ¿Qué hace `not`?
- 12. ¿Qué pregunta responde "CNC-01" in máquinas?
- 13. ¿Qué valor produce $2 + 3 * 4$?
- 14. ¿Qué valor produce $(2 + 3) * 4$?
- 15. ¿Por qué `is` no es el sustituto general de $==$?
- 16. ¿Qué error ocurre al dividir entre cero?
- 17. Escribe una condición que indique que temperatura supera límite.
- 18. Escribe una condición que exija motor activo y puerta cerrada.
- 19. ¿Qué familia de operadores usarías para buscar una máquina dentro de una lista?
- 20. Explica la cadena entrada $\rightarrow$ variable $\rightarrow$ operador $\rightarrow$ resultado $\rightarrow$ condición.

21. Respuestas y explicaciones

1. El operador indica la operación; el operando es el valor sobre el que actúa.
2. La expresión es lo que Python evalúa; el resultado es el valor obtenido.
3. 2.5, porque / realiza división normal.
4. / produce una división normal; // produce la división entera.
5. Devuelve el resto. Por ejemplo, 53 % 10 produce 3 piezas sobrantes.
6. Actualiza una variable sumándole el valor indicado; $x += 1$ equivale conceptualmente a $x = x + 1$.
7. True o False en las comparaciones habituales.
8. == compara valores; is comprueba identidad del objeto.
9. Cuando ambas condiciones deben cumplirse, por ejemplo sensor activo y temperatura segura.
10. Cuando basta con que al menos una condición sea verdadera.
11. Niega una expresión booleana.
12. Si la cadena "CNC-01" pertenece a la colección maquinas.
13. 14, porque * tiene mayor precedencia que +.
14. 20, porque los paréntesis fuerzan primero la suma.
15. Porque is comprueba identidad, no igualdad general de valores.
16. Un error de división por cero; debe prevenirse o gestionarse.
17. Por ejemplo: temperatura > limite.
18. Por ejemplo: motor_activo and puerta_cerrada.
19. in o not in.
20. Se reciben datos; se almacenan; un operador los calcula/compara; se obtiene un resultado; ese resultado puede convertirse en una condición para decidir.

22. Resumen del capítulo

Concepto	Para qué sirve	Cuándo usarlo	Error común
Aritméticos	Calcular.	Operaciones numéricas.	Confundir /, // y %.
Asignación	Crear/actualizar.	Guardar estado.	Confundir = con ==.
Comparación	Generar condiciones.	Evaluar relaciones.	Usar operador equivocado.
Lógicos	Combinar condiciones.	Reglas múltiples.	Confundir and con or.
Pertenencia	Comprobar contenido.	Buscar en colecciones/secuencias.	Confundir in con ==.
Identidad	Comprobar mismo objeto.	Casos como is None.	Usarlo para igualdad.
Bit a bit	Manipular bits.	Bajo nivel y máscaras.	Aplicarlo sin entender binario.
Precedencia	Controlar orden.	Expresiones múltiples.	No aclarar intención.

Ideas clave que debes recordar

- Los operadores transforman y evalúan datos.
- Una expresión produce un valor que puede almacenarse en una variable.
- = asigna y == compara.
- / y // tienen resultados distintos; % devuelve el resto.
- Las comparaciones generan condiciones.
- and, or y not permiten construir lógica.
- in comprueba pertenencia; is comprueba identidad.
- Los paréntesis ayudan a controlar el orden y comunicar intención.
- Los casos límite deben probarse.
- La claridad de las expresiones es parte de la calidad profesional.

23. Mapa de aprendizaje

Etapas	Progresión
Conocimiento previo	Variables + tipos + entrada/salida
Nuevo concepto	Operadores y expresiones
Aplicación básica	Cálculos y comparaciones
Aplicación práctica	Sensores, inventarios y producción
Integración	Condicionales, bucles y funciones
Uso profesional	Reglas de negocio, automatización y sistemas

Cómo prepara este capítulo los siguientes

Ahora puedes convertir datos en resultados y condiciones. El siguiente paso natural es aprender **if**, **elif** y **else**, donde las comparaciones y expresiones lógicas controlarán el flujo del programa. Después, los operadores de asignación y aritmética serán esenciales para bucles y funciones.

Cierre

La meta no es memorizar símbolos aislados. La meta es leer un problema, identificar sus datos, seleccionar la operación correcta, escribir una expresión clara y comprobar que funciona también en situaciones límite.